Pi-hole Ad Blocker

Revised & Rebuilt - Home Lab Portfolio Project

Bryan Sykes | Alexandria, VA | May 2026

Project Overview

Pi-hole is a network-wide DNS-based ad blocker that runs as a self-hosted service on a local network. Rather than installing browser extensions device by device, Pi-hole intercepts DNS queries at the network level if a requested domain matches a blocklist entry, Pi-hole returns nothing, preventing the ad or tracker from ever loading.

This project represents a revisit and full rebuild of a previous Pi-hole deployment that was running but not functioning correctly. The original implementation had been installed approximately three months prior and showed a running container with zero DNS queries processed, meaning no devices on the network were actually routing traffic through it. This document covers the full diagnostic process, the mistakes encountered along the way (including one that caused a network outage), and the corrected approach that resulted in a fully operational deployment.

Environment

- OS: Windows 11, running on a mid-range home lab machine (8–16GB RAM)
- Network: Ethernet-only connection, static DHCP reservation via router at 192.168.1.182
- Platform: Docker Desktop for Windows (no Linux VM, no VirtualBox)
- Pi-hole Version: v4.61.0
- Router DNS: Pre-configured to point to 192.168.1.182 prior to this session

Problem Statement — What Wasn't Working

The original Pi-hole container (named: unruffled_gould) was running and appeared healthy in Docker Desktop. However, inspecting the container logs revealed the root issue immediately:

```
INFO: -> Total DNS queries: 0
INFO: -> Blocked DNS queries: 0
INFO: -> Unique clients: 0
```

Zero queries meant zero devices were routing DNS through Pi-hole. The container was alive, but completely isolated, nothing on the network was talking to it. Two root causes were identified:

- **1.** Incorrect UDP port mapping - The container showed port 52:53/UDP instead of 53:53/UDP. Since the vast majority of DNS traffic uses UDP, this single typo meant DNS queries physically could not reach Pi-hole even if devices were pointed at it.
- **2.** Missing DNS routing The router DNS setting had never been verified as actually directing device traffic to Pi-hole's IP address.

Figure 1 — Original Broken Container (Zero Queries, Wrong UDP Port)

Docker Desktop showing the original Pi-hole container with 0 DNS queries processed and the misconfigured 52:53 UDP port mapping.

What Was Done Differently

1. Switching from GUI Setup to Docker Compose

The original container was created through the Docker Desktop GUI while following a video tutorial, which introduced the port typo. The corrected approach used a docker-compose.yml file created directly in PowerShell, providing a repeatable, auditable configuration that eliminates manual entry errors.

Critically, ports were bound to the specific local IP address (192.168.1.182) rather than 0.0.0.0 (all interfaces). This approach sidesteps Windows port conflicts without requiring any system service modifications:

```
ports:
  - "192.168.1.182:53:53/tcp"
  - "192.168.1.182:53:53/udp"
  - "192.168.1.182:80:80/tcp"
```

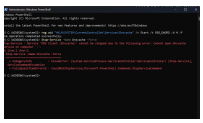
2. The Dnscache Incident — A Documented Mistake

An initial attempt to free port 53 involved disabling the Windows DNS Client service (Dnscache) via a registry edit. While this is a commonly cited approach online, it caused an immediate and complete loss of Ethernet connectivity upon restart. Windows relies on Dnscache for more than just DNS caching, and disabling it entirely is too aggressive for a Docker Desktop environment.

Recovery required booting into Safe Mode and reverting the registry change:

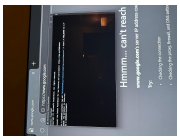
```
reg add "HKLM\SYSTEM\CurrentControlSet\Services\Dnscache" /v Start /t REG_DWORD
/d 2 /f
```

Figure 2 — Dnscache Disable Attempt and Error



PowerShell showing the registry edit that disabled Dnscache. The Stop-Service error was expected and harmless — but the registry change caused a full network outage on reboot.

Figure 3 — Network Outage Caused by Dnscache Disable



The resulting network outage: browser showing DNS resolution failure while a recovery command runs in PowerShell. Note the typo in the first recovery attempt (SYSYEM vs SYSTEM) requiring a second attempt.

The correct solution was binding Pi-hole's ports to the specific LAN IP rather than 0.0.0.0, which completely avoids the port 53 conflict with Dnscache. No system service modifications are needed when using IP-specific port binding in Docker.

Diagnostic Status at Resolution

Step	Status	Result
Pi-hole container running	✅ Done	
DHCP reservation in router	✅ Done	
Router DNS pointed to 192.168.1.182	✅ Done	
UDP port 53 mapped correctly	⚠️ Fixed (was 52:53)	
Devices querying Pi-hole	✅ Active - 517 queries (at the time of documentation)	

Verification

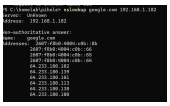
DNS Resolution Test

Before touching any router settings, Pi-hole's DNS response was verified directly using nslookup:

```
nslookup google.com 192.168.1.182
```

A successful response confirmed Pi-hole was listening on the correct IP and port, resolving DNS queries, and ready to handle network traffic before any router changes were made.

Figure 4 — Successful nslookup Verification



nslookup confirming Pi-hole at 192.168.1.182 successfully resolved google.com — proving the container was operational before any router configuration was touched.

Outcome

With the corrected port mapping and verified DNS routing, the Pi-hole dashboard immediately began showing live traffic from network devices. Within minutes of the container starting, two active clients were detected and DNS queries were being processed and blocked in real time.

Figure 5 — Live Pi-hole Dashboard

Pi-hole dashboard showing 517 total queries, 23 blocked, 4.4% block rate, and 82,208 domains across loaded blocklists — all within minutes of the corrected deployment.

Final initial metrics at time of documentation:

- Total DNS queries processed: 517
- Queries blocked: 23 (4.4%)
- Active clients: 2
- Domains on blocklists: 82,208
- Pi-hole version: 4.61.0
- Container status: Running, restart policy set to unless-stopped

Skills Demonstrated

- **Docker Desktop on Windows** — container creation, management, and compose file authoring
- **DNS fundamentals** — understanding of TCP vs UDP, port 53, nslookup verification methodology
- **Network troubleshooting** — diagnosing zero-query Pi-hole, identifying misconfigured port mappings
- **Incident response** — recovering from a self-inflicted network outage via Safe Mode registry revert
- **Iterative problem solving** — recognizing a flawed approach (Dnscache disable) and pivoting to a cleaner solution (IP-bound ports)
- **Documentation** — capturing a real troubleshooting session with screenshots, commands, and lessons learned

Lessons Learned

- Always use docker-compose.yml over GUI container creation; it's auditable, repeatable, and avoids manual typos in port mappings.
- Bind Docker ports to a specific LAN IP rather than 0.0.0.0 when running on Windows. This avoids system-level conflicts without touching protected services like Dnscache.
- Verify DNS resolution with nslookup before assuming router configuration is the problem. Isolating the container first saves time.
- Disabling Dnscache on Windows is too aggressive for a Docker Desktop setup; it breaks networking at the OS level.
- DHCP reservation in the router is the correct way to maintain a stable IP on Windows, not a static IP set in the OS network adapter settings.

Next Steps

- Add Hagezi Pro and Firebog blocklist to increase block rate beyond the current 4.4%
- Configure Docker Desktop to start on login so Pi-hole persists across reboots without manual intervention
- Monitor dashboard over 48–72 hours to assess real-world block rate across all household devices
- Proceed with Pelican game server setup (Minecraft + Palworld) as a parallel home lab project on the same Docker Desktop instance

— End of Document —